

Tableaux: recherche et tri

(DUT1-L1/DGI/ESP/UCAD)

Pr Ibrahima FALL

Ibrahima.Fall@esp.sn

Département Génie Informatique (DGI), Ecole Supérieure Polytechnique (ESP)

Université Cheikh Anta Diop (UCAD) de Dakar

BP 5085 Dakar-Fann, Sénégal

Objectifs

■ Objectif général

- Savoir gérer les éléments d'un tableau

■ Objectifs spécifiques

- OS1. Comprendre le déroulement des principaux algorithmes de recherche dans et de tri dans un / de tableaux
- OS2. Savoir mettre en œuvre les algorithmes vus avec l'aide de programmes

PLAN

1. **Rappels sur les tableaux**
2. **Recherche d'un élément**
3. **Tri à bulles, tri par selection et tri par insertion**
4. **Tri rapide et tri par fusion**

BIBLIOGRAPHIE

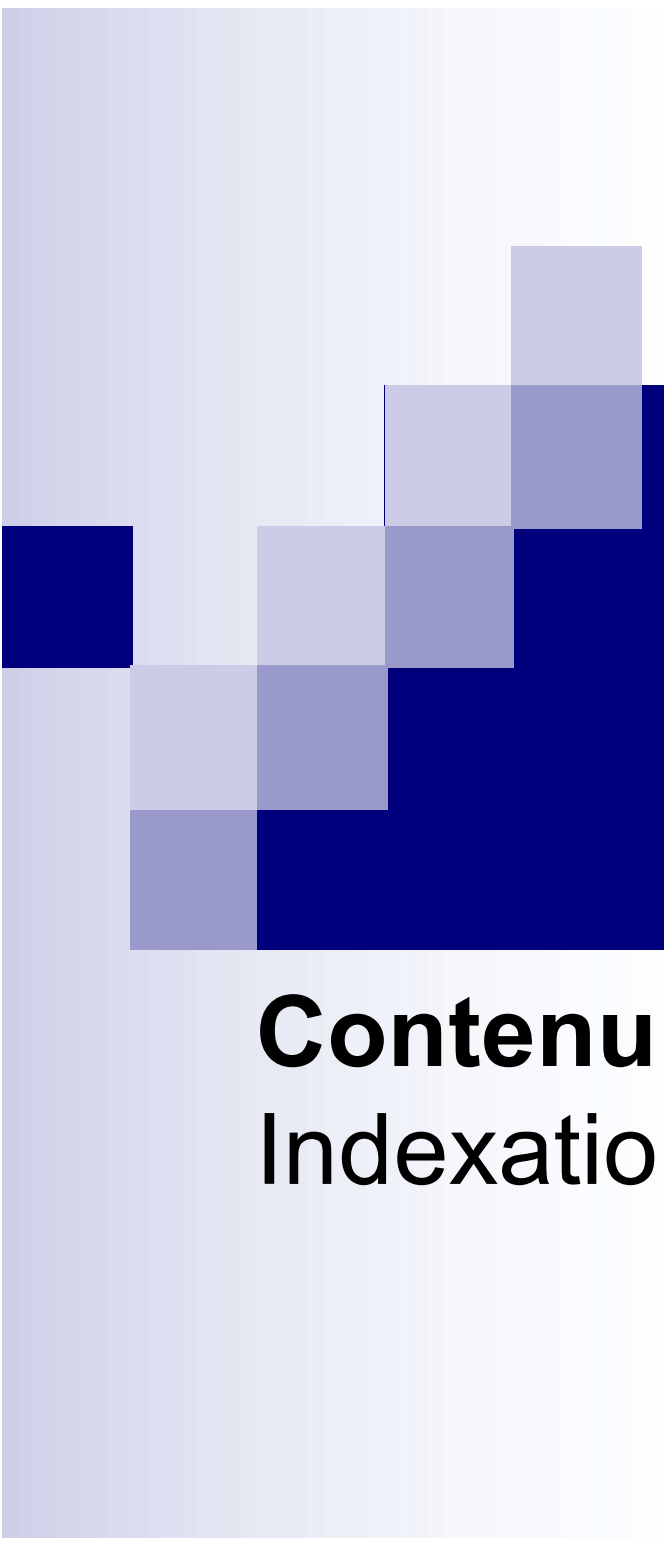
- Meyer B. et C. Baudouin, *Méthodes de programmation*, Eyrolles
- Clavel G et F.B Jorgensen, *Introduction à la programmation*, tome 3 Masson
- Wirth N, *Algorithmes et structures de données*, Eyrolles
- Richard C. et P., *Initiation à l'algorithmique*, Dia
- Mohamed Naimi, *Introduction à la programmation*, Hermes
- Le langage C : norme ANSI. Brian W. Kernighan & Denis M. Ritchie, Dunod
- Programmer en langage C. Claude Delannoy, Eyrolles
- C : a reference manual. Samuel P. Harbison & Guy L. Steele Jr., Prentice Hall
- C : a software engineering approach. Peter A. Darnell & Philip E. Margolis, Springer
- L'essentiel des structures de données en C. Ellis Horowitz, Sartaz Sahni & Susan Anderson-Freed
- S'initier à la programmation. Claude Delannoy, Eyrolles
- C-précis et concis. Peter Printz & Ulla Kirch-Printz, Oreilly

EVALUATION

- 33% Contrôle continu
 - ☐ Des TP's évalués
 - ☐ Des interrogations écrites
 - ☐ (Un contrôle)
 - ☐ Un projet
- 67% Examen
 - ☐ Sur papier

ORGANISATION

- 16h de cours
 - 8 séances de 2h
- 26h de TD/TP
 - 13 séances de 2h
- Au nom du respect mutuel [J'INSISTE]
 - Être en classe à l'heure prévue (voir l'emploi du temps)
 - Prière de ne pas rentrer en classe une fois le cours commencé
 - Se garder de manger/boire en classe
 - Ne pas être l'auteur de dérangements sonores (portables, bruits de machines, voix, etc.)
 - Se passer de son téléphone et des réseaux pendant les séances de CM/TD/TP



Chapitre 1

Rappel sur les tableaux

Contenu: Définition – Déclaration -
Indexation

Tableau à 1 dimension

- Un tableau est une collection ordonnée d'éléments (appelées composantes du tableau) ayant tous le même type.
 - Structurer les données est souvent indispensable pour les manipuler dans les programmes.
 - Exemples: les notes des étudiants sur une matière; la liste des clients d'une entreprise, ...
- On accède à chacun de ces éléments individuellement à l'aide d'un indice.
- L'indice
 - Valeur ordinale, c'est à dire ayant un domaine de valeurs finies et ordonnées.
 - Doit être de type scalaire.
 - indique la position relative de la composante dans la suite et permet donc d'accéder à la composante.

Déclaration d'un tableau

- S'effectue en donnant :
 - son nom (identificateur de la variable tableau)
 - le domaine de variation de l'indice délimité par une borne inférieure correspondant à l'indice minimal et la borne supérieure correspondant à l'indice maximal
 - le type de ses composantes
- Syntaxe
 - **Variable** <Nom_tab> : **Tableau**[<indice_min> .. <indice_max>] **de** <type des composants>
 - En C
 - Type nomTab [<taille>;
- Exemple
 - Variable tab_entier : Tableau[1..10] d'entiers (en pseudo-code)
 - int Tab_entier[10]; (en C)

Indexation des valeurs d'un tableau

- S'effectue pour un tableau T :
 - Sous la forme **T[indice]**
 - Où *indice* est un entier positif compris entre les indices min et max (inclus) de T
- Exemples
 - Ecrire (T[4]);
 - En C
 - `printf("%d", T[x])` //x compris entre 0 et taille-1

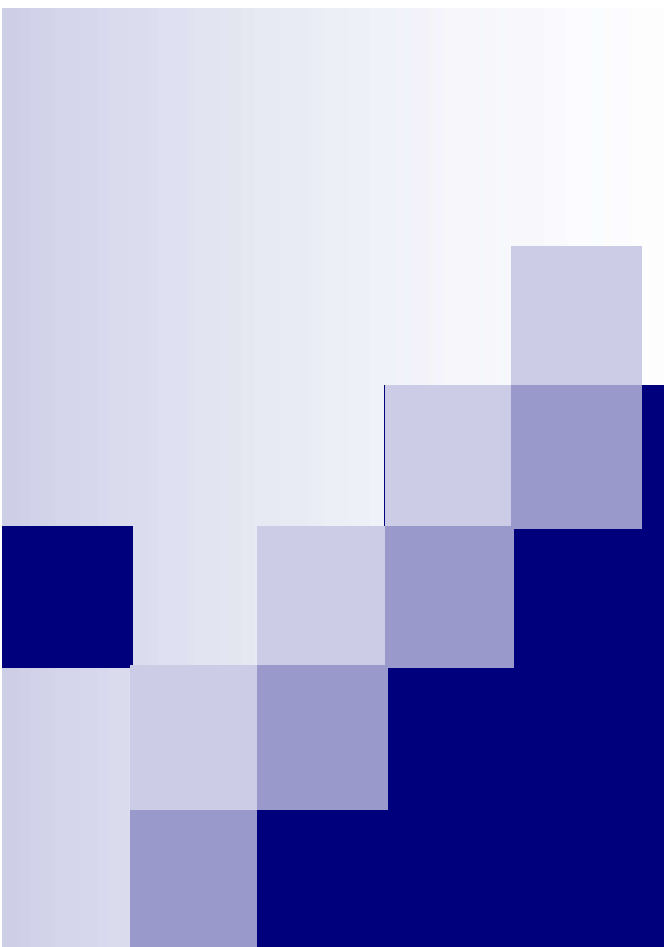
Les tableaux en mémoire

- En mémoire, les éléments d'un tableau sont rangés de façon contigüe
 - Une variable T de type tableau à une dimension de 10 entiers peut ainsi être représenté comme suit

1	2	3	4	5	6	7	8
10	0	1	4	7	2	6	100

Remarques sur les tableaux

- Quand on ne connaît pas à l'avance le nombre d'éléments exactement, il faut majorer ce nombre, quitte à n'utiliser qu'une partie du tableau.
- De façon générale il faut considérer qu'il n'y a pas un moyen automatique
 - de savoir la taille d'un tableau;
 - de savoir le nombre de cases utilisées d'un tableau.
- L'utilisation judicieuse des constantes permet de modifier aisément le programme et limite les sources d'erreurs.
- Il est interdit de mettre une variable dans l'intervalle de définition du tableau.



Chapitre 2

Recherche dans un tableau

Contenu: Recherche d'un élément -
Recherche séquentielle - Recherche
dichotomique

Recherche d'un élément

- Le problème est de trouver un élément dans une structure linéaire (tableau)
 - L'élément peut ne pas être présent
 - L'élément peut être présent à plusieurs endroits
 - Si la structure a plusieurs dimensions, il faut fouiller chaque dimension
- Plusieurs techniques de recherche existent
 - Recherche séquentielle
 - Recherche dichotomique
 - Etc.

Recherche séquentielle

■ Technique intuitive

- ☐ On parcourt la structure dans l'ordre « naturel »
- ☐ On s'arrête au bout, ou quand on trouve l'élément
- ☐ Parfois on peut s'arrêter plus tôt

Recherche séquentielle

```
1  fonction rechercheSequentielle(tab: tabChaines, e: chaine, n: entier): boolean
2  i: entier
3  trouve: boolean
4  debut
5  i <- 1;
6  trouve <- faux;
7  tantque (i <=n et non trouve) faire
8      si (tab[i] = e) alors
9          trouve <- vrai;
10     sinon
11         i <- i + 1;
12     finsi
13 fintantque
14 retourner trouve;
15 fin
```


Recherche séquentielle

- Variante: on peut chercher le nombre d'occurrences

```
1  fonction rechercheNbOccurrences(chaine: tabChaines, e: chaine, n: entier): entier
2  entier i, nbOc;
3  debut
4      i <- 1;
5      nbOc <- 0;
6      tantque (i <= n) faire
7          si (tab[i] = e) alors
8              nbOc <- nbOc + 1;
9          finsi
10         i <- i + 1;
11     fintantque
12     retourne nbOc;
13 fin
```

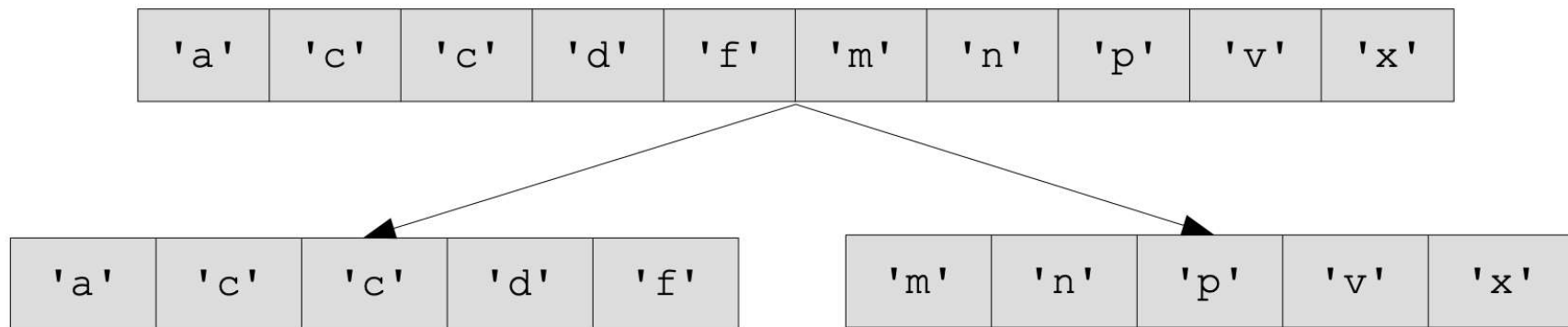
Recherche séquentielle

- Optimisation: si le tableau est trié, on peut s'arrêter plus « tôt »!

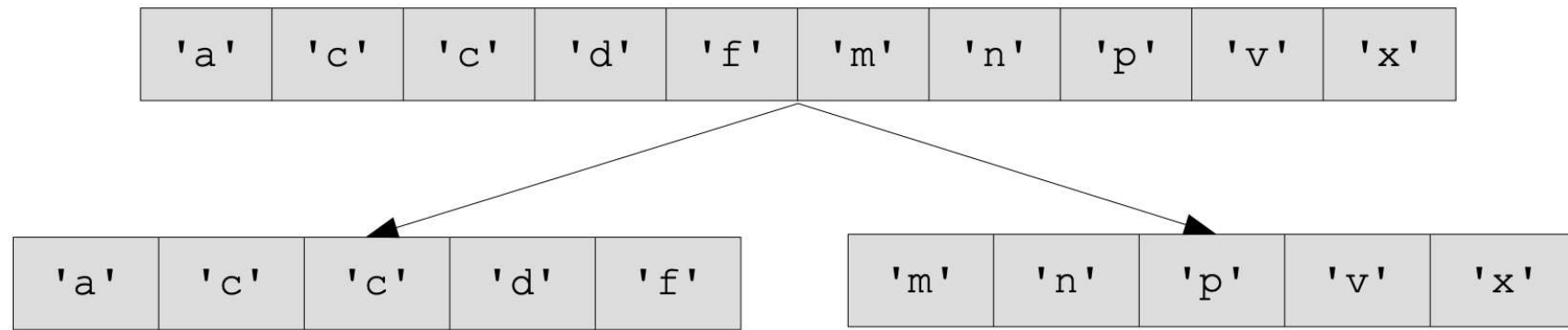
```
1  fonction rechercheSequentielleAmelioree(tab: tabChaines, e: chaine, n: entier): boolean
2  entier i;
3  trouve,possible: boolean;
4  debut
5      i <- 1;
6      trouve <- faux;
7      possible <- vrai;
8      tantque (i <= n et possible et non trouve) faire
9          si (tab[i] = e) alors
10             trouve <- vrai;
11         sinon
12             si (tab[i] > e) alors
13                 possible <- faux;
14             sinon
15                 i <- i + 1;
16             finsi
17         finsi
18     fintantque
19     retourne trouve;
20 fin
```

Optimisation de la recherche

- Remarque : il n'y a optimisation que si on trouve « assez vite » un point d'arrêt
- Principe **diviser pour régner** : on découpe les données en différentes parties qu'on traite séparément.
- Exemple
 - pour rechercher un élément dans un tableau, on le coupe en deux et on cherche dans chaque partie.



Optimisation de la recherche



■ Ce principe est intéressant

- Si on peut lancer des programmes en parallèle, on mettra environ deux fois moins de temps pour rechercher l'élément.
- **Si le tableau est trié, on peut ne chercher l'élément que dans une seule des parties.**

Recherche dichotomique

- Encore appelée recherche par dichotomie
- Principe de l'algorithme (recherche d'un élément e dans un tableau t)
 1. On regarde l'élément situé au milieu de t :
 2. S'il s'agit de e , c'est gagné.
 3. S'il est plus grand que e , on cherche dans la moitié gauche.
 4. S'il est plus petit que e , on cherche dans la moitié droite.
- Remarques
 - ☐ Ça ne peut marcher que **si le tableau est trié**
 - ☐ On coupe le tableau en deux parties pas forcément égales en taille

Recherche dichotomique

- Exemple: rechercher 490 dans le tableau d'entiers suivant

□ On trouve la valeur en **4 itérations**, une recherche séquentielle aurait demandé **11 itérations**

4	17	25	26	45	45	87	102	234	237	490	121 3	568 1	569 0	701 2
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
4	17	25	26	45	45	87	102	234	237	490	121 3	568 1	569 0	701 2
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
4	17	25	26	45	45	87	102	234	237	490	121 3	568 1	569 0	701 2
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
4	17	25	26	45	45	87	102	234	237	490	121 3	568 1	569 0	701 2
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Recherche dichotomique

```
1 // le tableau tab est suppose trié par ordre croissant
2 fonction rechercheDichotomique(tab: tabChaines, e: chaine, n: entier): boolean
3 entier i, j;
4 boolean trouve;
5 debut
6     trouve <- false;
7     i <- 1;
8     j <- n;
9     tantque (i <= j et non trouve) faire
10         si (tab[(j+i)/2] = e) alors
11             trouve <- vrai;
12         sinon
13             si (tab[(j+i)/2] > e) alors
14                 j <- (j+i)/2 - 1;
15             sinon
16                 i <- (j+i)/2 + 1;
17             finsi
18         finsi
19     fintantque
20     retourne trouve;
21 fin
```

Concept de récursivité

■ Exemples

□ Le tableau de tableaux

- Cela suppose que les tableaux qui constituent le premier peuvent aussi être constitués de tableaux et ainsi de suite

□ Définition par récurrence (en mathématiques, on parlera plutôt de récurrence) d'un nombre entier naturel

- 0 est un entier naturel, si n est un entier naturel, alors $n + 1$ est aussi un entier naturel

□ Définition du factoriel

- $0! = 1$, pour tout n appartenant à $\mathbb{N}$, $n! = n(n-1)!$

Définition de la récursivité

- En termes informatiques, on dira qu'un objet est récursif
 - S'il se contient lui-même ou s'il est défini à partir de lui-même
- Une fonction récursive est **une fonction qui s'appelle elle-même dans sa définition.**
 - Une fonction peut s'appeler elle-même dans son déroulement
 - Il faut nécessairement une condition d'arrêt

Exemples avec la récursivité

■ Calcul du factoriel

```
1  fonction factoriel(n: entier): entier
2      DEBUT
3      si (n=0) alors renvoyer 1;    (*la condition d'arret*)
4      sinon renvoyer (n*factoriel(n-1));    (*l'appel récursif*)
5      FIN
-
```

Exemples avec la récursivité

■ Nombre de Fibonacci

$$\square \text{Fibo}(0) = \text{Fibo}(1) = 1$$

$$\square \text{Fibo}(n) = \text{Fibo}(n-1) + \text{Fibo}(n-2);$$

```
1  fonction fibo(n: entier):entier
2  |  DEBUT
3  |      si (n<=1) renvoyer 1;    (* la condition d arret *)
4  |      sinon renvoyer(fibo(n-1)+fibo(n-2));    (* les appels ré2cursifs *)
5  |  FIN
```

Programmes itératifs vs programmes récursifs

- La récursivité simplifie la structure d'un programme ...
- ... Mais un programme risque d'être beaucoup plus performant si l'on en élimine les appels récursifs
 - La plupart du temps, le gain en simplicité vaut bien une baisse relative des performances d'exécution !!!

Programmes itératifs vs programmes récursifs

- La récursivité nécessite l'emploi de techniques spéciales de compilation
 - Le concept de **pile**
- Ces techniques sont généralement plus coûteuses en temps d'exécution que celles fondées sur l'itération
- Ainsi il arrive que l'on souhaite éliminer la récursivité dans les parties d'un programme les plus fréquemment exécutées
 - Il faut noter que l'on peut montrer qu'il est toujours possible de transformer un sous-programme itératif en un sous-programme récursif;
 - **Cependant, la réciproque n'est pas vraie.!**
 - On peut donc dire que la classe des problèmes récursifs contient celle des problèmes itératifs.

Récurtivité : exercices

- Donner les versions itératives des deux fonctions **factoriel** et **fibonacci**
- Donner la version récursive de la fonction de recherche dichotomique



Chapitre 3

Tri de tableaux

Contenu: Tableau trié - Algorithmes de tri
- Tri à bulles - Tri par sélection - Tri par insertion séquentielle - Tri rapide - Tri par fusion

Tableau trié

- La recherche d'éléments paraît plus efficace dans les tableaux triés
 - Il est ainsi bien de pouvoir tester si un tableau est trié
 - Et de pouvoir le trier sinon
- Pour tester si un tableau est trié par ordre croissant (ou décroissant)
 - On s'assure que chaque case (sauf la dernière) a un contenu plus petit (ou plus grand) que celui de la case suivante

Tableau trié

- Tester si un tableau est trié par ordre croissant
 - On s'assure que chaque case (sauf la dernière) a un contenu plus petit que celui de la case suivante

```
1  fonction testTrie1( tab: tableauEntiers, n: entier): boolean
2      i: entier;
3      b: boolean;
4  debut
5      b <- VRAI;
6      pour (i allant de 1 à n-1 pas 1) faire
7          si (tab[i] > tab[i+1]) alors
8              b <- FAUX;
9          finsi
10     finpour
11     retourne b;
12 fin
```

Tableau trié

- Tester si un tableau est trié par ordre croissant
 - Optimisation: on peut s'arrêter dès qu'on trouve deux éléments qui ne sont pas dans le bon ordre!

```
1  fonction testTrie2(tab: tableauEntiers, n: entier): boolean
2      i: entier;
3      b: boolean;
4  debut
5      b <- VRAI;
6      i <- 1;
7      tantque (i < n et b) faire
8          si (tab[i] > tab[i+1]) alors
9              b <- FAUX;
10         finsi
11         i <- i+1;
12     fintantque
13     retourne b;
14 fin
```

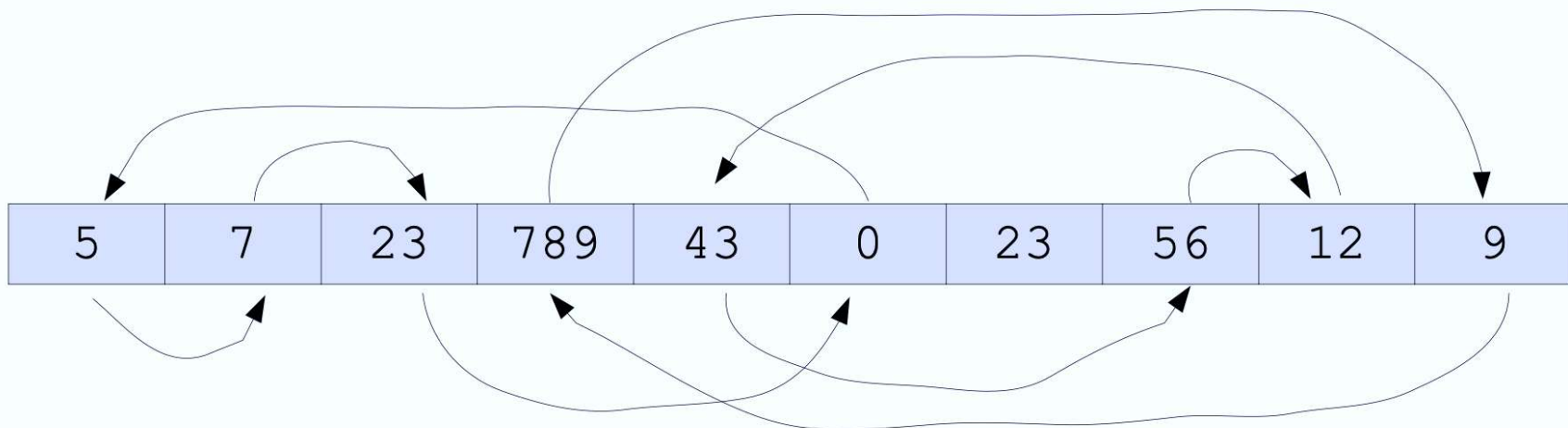
Problème du tri et algorithmes

- Placer les éléments d'une structure linéaire dans l'ordre
 - ☐ les éléments doivent pouvoir être comparés
 - ☐ on peut trier par ordre croissant ou décroissant
 - ☐ si la structure a plusieurs dimensions, on peut trier chaque dimension
- De nombreux algorithmes de tri existent
 - ☐ Chacun d'eux a ses avantages et inconvénients en fonction des données à trier
 - ☐ Certains algorithmes utilisent des structures de données plus complexes (arbres en particulier)
 - ☐ Dans ce cours nous n'allons traiter que certains algorithmes de tri de base

Algo de tri: **occupation mémoire**

■ Tri **en place**

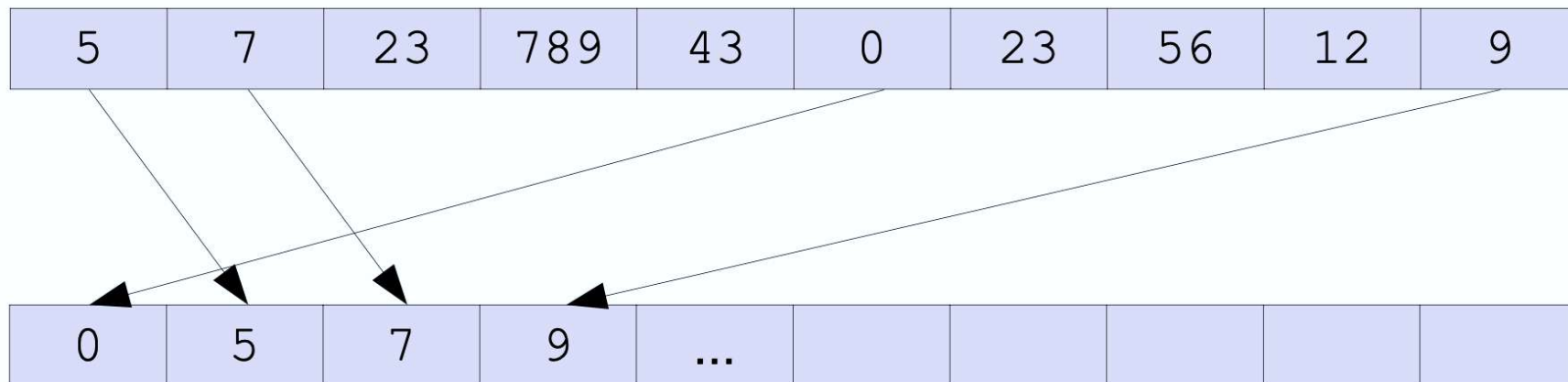
- les éléments sont triés dans la structure de données elle-même



Algo de tri: **occupation mémoire**

■ Tri **pas en place**

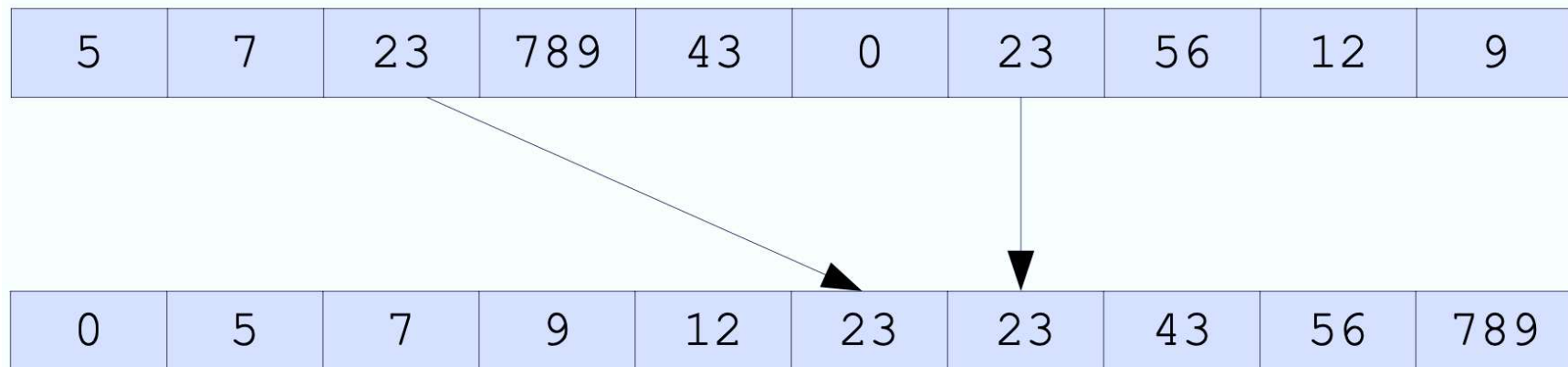
- on crée une nouvelle structure de données



Algo de tri: **stabilité**

■ Tri **stable**

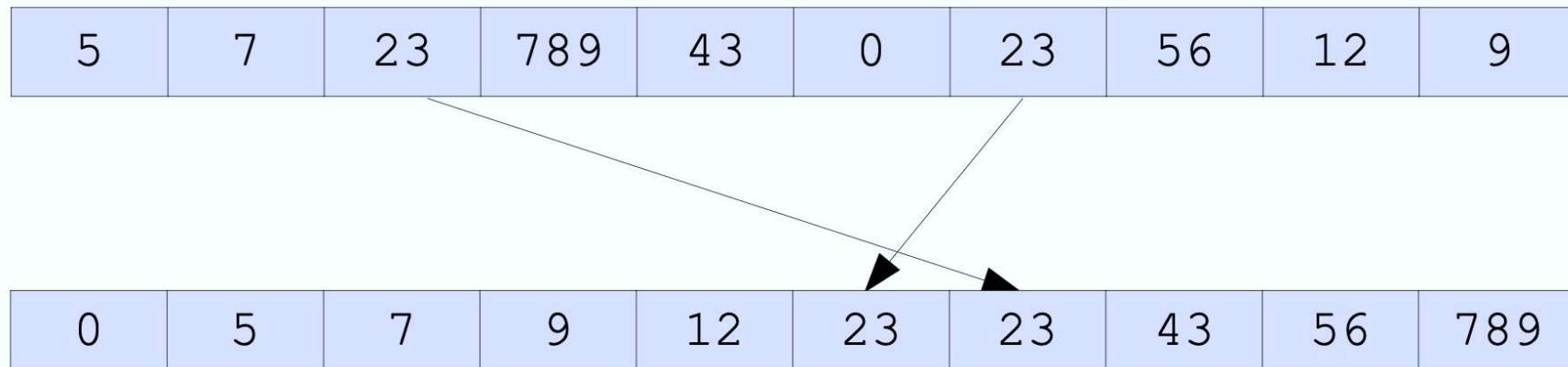
- l'algorithme ne modifie pas l'ordre initial des éléments égaux



Algo de tri: **stabilité**

■ Tri **instable**

- l'algorithme modifie l'ordre initial des éléments égaux



Tri à bulles

- On remonte le plus grand élément par permutations et on recommence jusqu'à ce que le tableau soit trié
- Exemple : tri par ordre croissant d'un tableau

17	2	268	415	45	45	102	701
----	---	-----	-----	----	----	-----	-----

2	17	268	45	45	102	415	701
---	----	-----	----	----	-----	-----	-----

2	17	45	45	102	268	415	701
---	----	----	----	-----	-----	-----	-----

2	17	45	45	102	268	415	701
---	----	----	----	-----	-----	-----	-----

Tri à bulles

- Le tri à bulles est en place
- Il est stable si on permute uniquement les éléments différents
- On pourrait faire remonter le plus petit élément
- On pourrait s'arrêter dès que le tableau est trié

Tri à bulles

- Tri d'un tableau d'entier, par ordre croissant

- Ce tri est stable,

- il serait non stable si on avait mis **tab[j] >= tab[j+1]**

```
1  procedure triBulle(tab: tableauEntiers, n: entier)
2  |   i,j,temp: entier;
3  debut
4  |   pour (i allant de n-1 a 2 pas -1) faire
5  |   |   pour (j allant de 1 a i pas 1) faire
6  |   |   |   si (tab[j] > tab[j+1]) alors
7  |   |   |   |   temp <- tab[j];
8  |   |   |   |   tab[j] <- tab[j+1];
9  |   |   |   |   tab[j+1] <- temp;
10 |   |   |   finsi
11 |   |   finpour
12 |   finpour
13 fin
```

Tri à bulles optimisé

- On s'arrête dès que le tableau est trié
 - Cette amélioration n'est efficace que si le tableau est déjà « un peu » trié et qu'il devient trié « assez vite »

```
1  procedure triBulleOptimise(tab: tableauEntiers, n: entier)
2      i,j,temp: entier;
3      trie: boolean;
4      debut
5          trie <- faux;
6          i <- n-1;
7          tantque ((non trie) et (i > 1)) faire
8              trie <- vrai;
9              pour (j allant de 1 à i pas 1) faire
10                 si (tab[j] > tab[j+1]) alors
11                     temp <- tab[j];
12                     tab[j] <- tab[j+1];
13                     tab[j+1] <- temp;
14                     trie <- faux;
15                 finsi
16             finpour
17             i <- i-1;
18         fintantque
19     fin
```

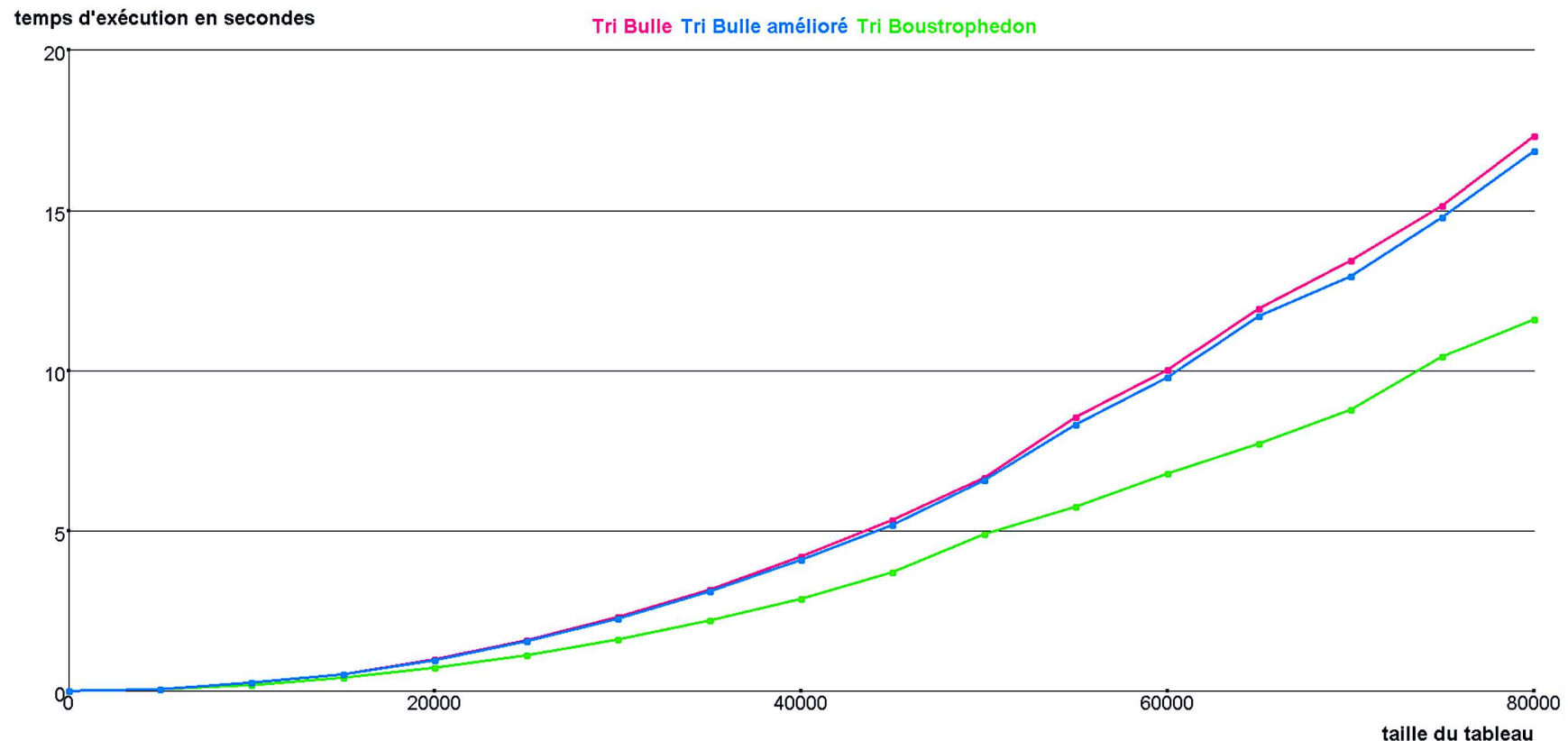
Tri à bulles boustrophédon

- A chaque itération, on remonte le plus grand élément à un bout et le plus petit élément à l'autre bout

```
1  procedure triBulleBoustrophedon(tab: tableauEntiers, n: entier)
2      i,j,k,temp: entier;
3      trie: boolean;
4      debut
5          trie <- faux; j <- n+1; i <- 0;
6          tantque ((non trie) et (j > i)) faire
7              trie <- vrai;
8              pour (k allant de i+1 a j-2 pas 1) faire
9                  si (tab[k] > tab[k+1]) alors
10                     temp <- tab[k]; tab[k] <- tab[k+1]; tab[k+1] <- temp;
11                     trie <- faux;
12                 finsi
13             finpour
14             j <- j-1;
15             pour (k allant de j-1 a i+2 pas -1) faire
16                 si (tab[k] < tab[k-1]) alors
17                     temp <- tab[k]; tab[k] <- tab[k-1]; tab[k-1] <- temp;
18                     trie <- faux;
19                 finsi
20             finpour
21             i <- i+1;
22         fintantque
23     fin
```

Temps d'exécution des algo de tri à bulles

- Test expérimental sur des tableaux d'entiers générés aléatoirement



Tri par sélection

- On parcourt le tableau pour trouver le plus grand élément, et une fois arrivé au bout du tableau on y place cet élément par permutation. Puis on recommence sur le reste du tableau

```
1  procedure triSelection(tab: tableauEntiers, n: entier)
2      variables i,j,temp,pg: entier;
3      debut
4          i ← n;
5          tantque (i > 1) faire
6              pg ← 1;
7              pour (j allant de 1 à i pas 1) faire
8                  si (tab[j] > tab[pg]) alors
9                      pg ← j;
10             finsi
11             finpour
12             temp ← tab[pg];
13             tab[pg] ← tab[i];
14             tab[i] ← temp;
15             i ← i-1;
16         fintantque
17     fin
```

Tri par selection: caractéristiques

- Le tri sélection n'effectue qu'une permutation pour remonter le plus grand élément au bout du tableau
 - Il est donc plus efficace que le tri à bulle
- Le tri sélection est en place et l'algorithme donné ici est stable.
 - Il aurait été instable si on avait remplacé la comparaison **$\text{tab}[j] > \text{tab}[pg]$** par **$\text{tab}[j] \geq \text{tab}[pg]$**
- Le tri sélection peut être amélioré en positionnant à chaque parcours du tableau le plus grand et le plus petit élément
 - Selon le même principe que celui utilisé pour le tri à bulle boustrophédon

Tri par insertion

- On prend deux éléments qu'on trie dans le bon ordre, puis un 3e qu'on insère à sa place parmi les 2 autres, puis un 4e qu'on insère à sa place parmi les 3 autres, etc.

```
1  procedure triInsertion(tab: tableauEntiers, n: entier)
2      variables i, j, val: entier;
3      debut
4          pour (i allant de 2 a n pas 1) faire
5              val <- tab[i];
6              j <- i;
7              tantque ((j > 1) et (tab[j-1] > val)) faire
8                  tab[j] <- tab[j-1];
9                  j <- j-1;
10             fintantque
11             tab[j] <- val;
12         finpour
13     fin
```


Tri par insertion: caractéristiques

- L'algorithme de tri par insertion présenté ici est en place et stable
 - Il serait instable si on utilisait $\geq$ au lieu de $>$
- Le tri par sélection nécessite en moyenne de l'ordre de $n^2/2$ comparaisons, n étant la taille du tableau
 - On fera d'abord $n-1$ tours de la boucle pour, puis $n-2$, et ainsi de suite jusqu'à 1
- Le tri par insertion nécessite en moyenne de l'ordre de $n^2/4$ comparaisons et déplacements
 - Placer le i ème élément demande en moyenne $i/2$ comparaisons
- Le tri par insertion est donc environ deux fois plus rapide que le tri par sélection.

Temps d'exécution des algo de tri

- Test expérimental sur des tableaux d'entiers générés aléatoirement

